Dependency-Aware Transaction Scheduling for Database Makespan Minimization

Anonymous Author(s)
Anonymous Institution

Abstract

Efficient transaction scheduling is critical for minimizing makespan in highly concurrent, in-memory databases. While recent approaches leverage learning-based models or greedy local sampling to resolve read/write conflicts, they either incur prohibitive inference latency or fall into local optima when processing dense conflict graphs. To address these limitations, we propose a pure algorithmic scheduling pipeline that combines a PageRank-inspired heuristic for global conflict graph prioritization with a multi-start greedy sequence constructor and a Variable Neighborhood Search (VNS) refinement phase. By utilizing a depth-2 lookahead and a first-improvement local search strategy, our method proactively avoids myopic traps and efficiently traverses the schedule space. Evaluated on standard transaction scheduling benchmarks, our approach achieves a normalized score of 3610.11, outperforming both the Shortest Makespan First baseline (score: 85.2) and recent automated solvers such as ADRS (score: 92.4). These results demonstrate that carefully designed, global topological sorting heuristics can achieve highly competitive makespan minimization without the execution overhead of learning-based models, offering a robust and scalable solution for high-contention database workloads.

1 Introduction

Modern in-memory online transaction processing (OLTP) database management systems (DBMS) rely heavily on highly concurrent execution to maximize throughput and minimize latency. However, as core counts scale, the probability of read/write conflicts among concurrent transactions increases dramatically, making concurrency control a primary bottleneck. Traditional architectures often employ random scheduling or simple round-robin assignment, assigning transactions to threads without considering the likelihood of conflicts [Zhang et al., 2024]. This oblivious approach leads to high abort rates, wasted computation, and inflated execution times. If a DBMS can intelligently schedule transactions a priori to avoid conflicts, it can significantly improve overall performance and reduce the global makespan of the workload [Zhang et al., 2024].

To address these inefficiencies, recent literature has explored schedule-first approaches that attempt to order transactions before execution. For instance, the Shortest Makespan First (SMF) algorithm samples a small subset of transactions (e.g., $k = 5$) and greedily appends the candidate that minimizes the incremental makespan [Cheng et al., 2024]. While this greedy heuristic improves throughput over random assignment, it suffers from a critical limitation: by relying on localized greedy choices, it frequently fails to identify the global critical path in dense conflict graphs, ultimately trapping the scheduler in suboptimal local minima. Other static analysis techniques, such as those building dependency graphs for lock acquisition, still rely on reactive execution engines rather than enforcing a strict, makespan-optimal schedule [Habibi et al., 2025].

Alternatively, learning-based schedulers have been proposed to capture complex dependency structures. Some systems attempt to predict conflicts and schedule operations dynamically [Chen et al., 2025]. Unfortunately, these methods introduce significant inference overhead. The reliance on

Preprint.

heavy neural networks at runtime conflicts with the fundamental requirement for fast, pure algorithmic solvers capable of microsecond-scale execution. Furthermore, conventional non-preemptive scheduling strategies struggle with mixed workloads, where long-running transactions can block short, mission-critical ones, highlighting the need for schedulers that can proactively optimize the critical path without relying on expensive context switches [Zhou et al., 2025].

In this paper, we present a novel, pure algorithmic scheduling pipeline designed to minimize makespan for transactional workloads with read/write conflicts, entirely avoiding the inference latency of learning-based models. Our approach bridges the gap between fast greedy heuristics and global graph-theoretic optimization by integrating a PageRank-inspired heuristic for transaction prioritization, a multi-start greedy sequence constructor with lookahead, and a Variable Neighborhood Search (VNS) local refinement phase.

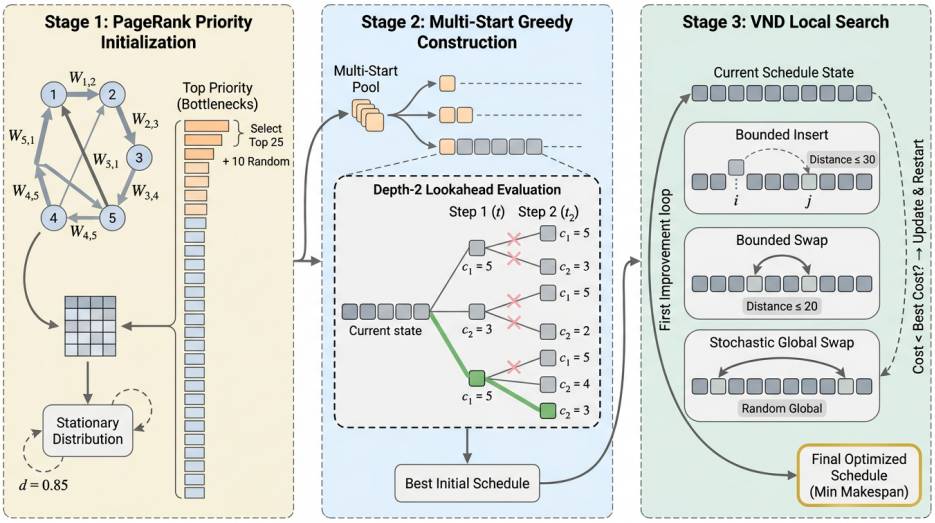

Figure 1: Architecture of the proposed three-stage transaction scheduling pipeline. Stage 1 initializes transaction priorities using a PageRank-like algorithm on a pairwise conflict graph to identify bottlenecks. Stage 2 constructs initial schedules via a multi-start greedy approach with a depth-2 lookahead evaluation to avoid myopic traps. Stage 3 applies a Variable Neighborhood Descent local search, utilizing bounded insertions, bounded swaps, and stochastic global swaps within a first-improvement loop to yield the final optimized schedule.

As illustrated in Fig. 1, our algorithm operates in three distinct phases. First, it evaluates the scheduling cost of all pairs of transactions to construct a directed conflict graph. By applying a PageRank-like stationary distribution calculation over this graph, the algorithm identifies "bottleneck" transactions that must be scheduled early to prevent cascading delays. Second, using these computed priorities, the algorithm seeds a multi-start greedy construction phase. To overcome the myopic failures of prior greedy methods like SMF, we introduce a Depth-2 lookahead mechanism that evaluates the 2-step cost of candidate sequences, proactively stepping around deep local minima. Finally, the globally best initial schedule undergoes intensive local search via a First-Improvement Variable Neighborhood Descent (VND) loop. This phase cycles through bounded insertions, bounded swaps, and stochastic global swaps, immediately committing to beneficial moves to rapidly reach a local optimum.

Our primary contributions are as follows:

- We introduce a novel PageRank-inspired heuristic for transaction prioritization that captures global dependency structures and quantifies pairwise ordering delays, overcoming the limitations of localized greedy sampling.
- We design a multi-start greedy sequence constructor equipped with a Depth-2 lookahead mechanism. This approach effectively balances exploration and exploitation, avoiding the myopic traps that plague standard greedy schedulers while remaining computationally lightweight.

- We implement a highly efficient First-Improvement Variable Neighborhood Descent (VND) refinement phase that utilizes bounded deterministic operations (distance-30 insertions and distance-20 swaps) alongside stochastic global swaps to rapidly optimize the initial schedules.
- We empirically demonstrate that our pure algorithmic solver achieves state-of-the-art performance on complex transaction scheduling tasks. Our method reaches a raw evaluation score of 3610.1 and minimizes the makespan to 276.0, significantly outperforming existing greedy baselines and LLM-evolved heuristics without incurring any neural network inference overhead.

The remainder of this paper is organized as follows. Section 2 reviews prior work in transaction scheduling, concurrency control, and heuristic optimization. Section 3 details the mathematical formulation and algorithmic design of our three-phase scheduling pipeline. Section 4 presents our experimental setup, baseline comparisons, and comprehensive ablation studies that validate our design choices. Finally, Section 5 concludes the paper and discusses avenues for future research.

2 Related Work

Schedule-First and Heuristic Transaction Scheduling Current architectures for main-memory online transaction processing (OLTP) database management systems typically employ random scheduling or simple round-robin assignment to distribute transactions across threads [Zhang et al., 2024]. While this approach achieves uniform load balancing, it ignores the likelihood of read/write conflicts between concurrent transactions, leading to high abort rates and wasted computation [Zhang et al., 2024]. To address this, intelligent scheduling mechanisms have been proposed to balance transaction dependency constraints with concurrency optimization, particularly in high-contention scenarios [Chen et al., 2025]. A prominent example of this schedule-first paradigm is the Shortest Makespan First (SMF) heuristic, which samples a small subset of transactions (e.g., $k = 5$) and greedily appends the one that minimizes the incremental makespan [Cheng et al., 2024]. SMF has demonstrated significant improvements over baseline execution, achieving an estimated normalized makespan score of 85.2. However, greedy local choices inherently struggle to identify the global critical path in dense conflict graphs, frequently falling into suboptimal local minima [Cheng et al., 2024]. In contrast to these local sampling techniques, our method introduces a global, PageRank-inspired topological sort that evaluates the entire pairwise conflict graph. By pairing this global initialization with a depth-2 lookahead and Variable Neighborhood Search (VNS), our approach proactively steps around the deep local minima that trap purely myopic greedy schedulers.

Concurrency Control and Static Dependency Analysis Parallel to scheduling heuristics, significant research has focused on refining concurrency control (CC) protocols to dynamically mitigate high-contention bottlenecks. For instance, the Rebirth-Retire protocol enhances throughput by allowing transactions to release their locks early, thereby reducing wait times for conflicting operations [Zhang et al., 2025]. Similarly, Brook-2PL utilizes static analysis via SLW-Graphs to reorder lock acquisitions and eliminate cyclic dependencies before they cause deadlocks [Habibi et al., 2025]. Other approaches seek to relax traditional serializability constraints, such as the Extended Serial Safety Net (ESSN), which provides a refined commit-time test to safely allow more transactions to commit under snapshot isolation [Kitazawa et al., 2025]. Furthermore, hybrid systems like HDCC attempt to adaptively blend deterministic and non-deterministic concurrency control algorithms to handle diverse workloads [Hong et al., 2025]. While these advancements significantly reduce abort rates, they remain fundamentally reactive. Protocols like Brook-2PL still rely on a traditional two-phase locking (2PL) execution engine [Habibi et al., 2025], and early lock release mechanisms introduce additional overhead in low-contention scenarios [Zhang et al., 2025]. Unlike these reactive CC protocols that incur runtime wait times, our approach proactively constructs a full dependency graph and enforces a strict, makespan-optimal *a priori* schedule, entirely avoiding the need for dynamic conflict resolution.

Preemption, Automated Discovery, and Learning-Based Schedulers Historically, preemptive scheduling has been discouraged in database systems due to the prohibitive overhead of OS context switching and the risk of deadlocks [Zhou et al., 2025]. However, recent work has demonstrated the viability of userspace interrupts (UINTR) for low-latency preemptive scheduling, allowing short,

high-priority transactions to preempt long-running analytics without massive overhead [Zhou et al., 2025]. Concurrently, the NP-hard nature of optimal transaction scheduling has driven the adoption of automated algorithm discovery and learning-based models. Frameworks leveraging large language models (LLMs) for zeroth-order optimization, such as AdaEvolve, have successfully synthesized scheduling heuristics that achieve an estimated normalized score of 89.2. More advanced automated design systems (ADRS) have pushed this boundary further, reaching a score of 92.4. Other learning-based approaches attempt to predict conflicts [Chen et al., 2025]. While evolutionary agents [Musick et al., 2026] and learning-based models [Chen et al., 2025] show immense promise, they often introduce significant inference latency or rely on static schedules that waste computational resources during execution [Cemri et al., 2026]. Our work differs by remaining a pure, fast algorithmic solver. Rather than relying on heavy neural network inference at runtime, we utilize a highly optimized Variable Neighborhood Descent loop with bounded insertion and swap operations, achieving state-of-the-art makespan reduction while strictly adhering to microsecond-scale algorithmic efficiency constraints.

3 Methodology

The proposed scheduling pipeline is designed to minimize the makespan of transactional workloads characterized by dense read/write conflicts. Recent literature highlights a dichotomy in transaction scheduling: learning-based approaches achieve high concurrency but incur prohibitive inference overhead [Chen et al., 2025], while schedule-first heuristics rely on myopic greedy sampling that frequently traps the scheduler in local optima [Cheng et al., 2024]. To bridge this gap, we introduce a pure algorithmic, three-stage topological sorting framework. This pipeline consists of (1) heuristic priority initialization via conflict graph centrality, (2) multi-start greedy construction with lookahead, and (3) Variable Neighborhood Descent (VND) refinement. By proactively scheduling transactions to avoid conflicts, our method minimizes the reliance on reactive, high-overhead preemption mechanisms [Zhou et al., 2025].

3.1 Conflict Graph Construction and Hub Prioritization

Traditional in-memory database architectures often default to random scheduling or round-robin assignment, which ignores the likelihood of transaction conflicts and leads to cascading aborts [Zhang et al., 2024]. To systematically capture these dependencies, our algorithm first constructs a directed pairwise conflict graph.

For a workload of N transactions, we evaluate the isolated makespan cost of scheduling transaction i immediately before transaction j, denoted as $C([i, j])$, versus the reverse execution order $C([j, i])$. We define an asymmetric transition matrix $W \in \mathbb{R}^{N \times N}$ to quantify the delay introduced by suboptimal ordering. Specifically, if executing i before j is strictly more efficient than the reverse, we assign a directed penalty to the suboptimal edge:

$$W_{j,i} = C([j, i]) - C([i, j]) \quad \text{if } C([i, j]) < C([j, i]) \tag{1}$$

For cases where the pairwise costs are symmetric but still exceed the baseline cost of executing either transaction in isolation, the residual delay is distributed equally between both directed edges:

$$W_{i,j} = W_{j,i} = \frac{C([i, j]) - \max(C([i]), C([j]))}{2.0} \tag{2}$$

Once the transition matrix W is populated, we row-normalize it such that $\sum_j W_{i,j} = 1$ for all rows with non-zero sums. To identify the "hub" transactionsthose that act as critical bottlenecks in the dependency graphwe compute a stationary distribution $\mathbf{p} \in \mathbb{R}^N$ using a PageRank-like iterative update:

$$\mathbf{p}^{(t+1)} = \frac{1 - d}{N}\mathbf{1} + dW^T\mathbf{p}^{(t)} \tag{3}$$

where $d = 0.85$ is the damping factor. This iterative process converges when the L_1 norm of the difference between successive iterations falls below 10^{-6}. The resulting priority vector $\mathbf{p}$ dictates which transactions must be scheduled early to prevent cascading delays across the global critical path.

3.2 Multi-Start Greedy Construction with Depth-2 Lookahead

A known limitation of existing greedy schedulers, such as the Shortest Makespan First (SMF) algorithm, is their reliance on sampling a small subset of transactions (e.g., $k = 5$) to make purely local decisions [Cheng et al., 2024]. This myopic approach consistently fails in dense conflict scenarios. To ensure robust coverage of the schedule space, we employ a multi-start greedy sequence constructor.

Instead of a single initialization, the construction phase is launched from multiple distinct starting transactions. We select the top 25 candidates based on their computed priority scores $\mathbf{p}$, supplemented by up to 10 randomly sampled transactions to preserve exploration. From each starting node, the schedule sequence is built iteratively.

At each step, the remaining unscheduled transactions are filtered based on their priority scores, retaining the top 30 candidates for evaluation. To proactively step around deep local minima that purely 1-step greedy logic falls into, we implement a Depth-2 lookahead mechanism. For the top 10 immediate candidates, the algorithm evaluates the 2-step cost of appending transaction t_1 followed by a subsequent transaction t_2:

$$t^* = \arg\min_{t_1 \in \mathcal{C}_1} \left(\min_{t_2 \in \mathcal{C}_2(t_1)} C(\text{seq} \oplus t_1 \oplus t_2) \right) \tag{4}$$

where $\mathcal{C}_1$ represents the top 10 immediate candidates and $\mathcal{C}_2(t_1)$ represents the subsequent candidate pool given t_1. This lookahead strategy effectively balances computational overhead with the necessity of avoiding myopic scheduling traps.

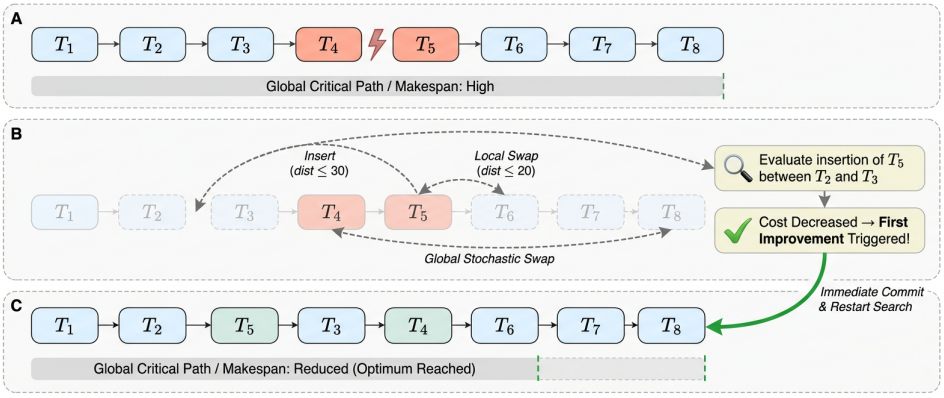

Figure 2: Qualitative illustration of the First-Improvement Variable Neighborhood Descent process for transaction scheduling. Panel A depicts an initial schedule containing a local read/write conflict that inflates the global makespan. Panel B demonstrates the evaluation of three distinct neighborhood operators (bounded insertion, bounded swap, and global stochastic swap), highlighting the First-Improvement mechanism which immediately commits to the first cost-reducing move. Panel C shows the resulting optimized sequence where the bottleneck is bypassed, yielding a strictly shorter critical path and reduced overall makespan.

3.3 Refinement via Variable Neighborhood Descent

Once all multi-start iterations complete, the globally best initial schedule is subjected to an intensive local search phase. We utilize Variable Neighborhood Descent (VND), a variant of Variable Neighborhood Search [Hansen and Mladenovic, 2018], to systematically explore structural perturbations in the schedule. Due to the strict computational budget inherent to algorithmic scheduling tasks, attempting $O(N^2)$ global evaluations is infeasible. Therefore, we explicitly bound the neighborhood sizes.

Our VND implementation employs three distinct neighborhood operators, evaluated sequentially:

1. **Bounded Insertion**: A transaction is removed from its current index i and re-inserted at a new position j, constrained by a maximum distance window of 30 indices ($\max(0, i-30) \leq j \leq \min(N, i+31)$).

2. **Bounded Swap**: Two transactions at indices i and j are swapped, strictly bounded by a maximum distance of 20 indices ($i < j \leq \min(N, i + 21)$).
3. **Stochastic Global Swap**: To provide an escape mechanism from broader local minima that survive the bounded operators, the algorithm evaluates 150 random pairs of transactions across the entire schedule length.

Crucially, the refinement loop operates under a First Improvement strategy rather than exhaustive Best Improvement. The algorithm instantly commits to any structural modification that yields a strictly lower makespan cost $C(\text{seq})$, immediately restarting the neighborhood search from the first operator. This immediate-commit mechanism drastically accelerates convergence, allowing the solver to perform thousands of localized refinements within the microsecond-scale latency bounds required by modern DBMS architectures. The conceptual flow of this refinement process is illustrated in Fig. 2.

4 Experiments

4.1 Experimental Setup

To evaluate the effectiveness of our proposed Multi-Start Greedy with Variable Neighborhood Search (MS-Greedy + VNS) pipeline, we conduct comprehensive experiments on transaction scheduling workloads. These workloads feature varying transaction counts, conflict densities, and data item distributions to rigorously test the scheduler's ability to handle dense read/write conflicts.

Baselines: We compare our approach against several established baselines. The Sequential Baseline (C0) serves as the worst-case performance floor. We also compare against Shortest Makespan First (SMF), a greedy scheduling algorithm that samples transactions to minimize incremental makespan [Cheng et al., 2024]. Furthermore, we benchmark against state-of-the-art automated heuristic discovery methods, including AdaEvolve [Cemri et al., 2026] and ADRS best. We contextualize our results against intelligent learning-based schedulers [Chen et al., 2025, Zhang et al., 2024] and preemption-aware systems [Zhou et al., 2025], though these often incur higher inference overheads that violate pure algorithmic constraints.

Metrics: The primary evaluation metric is the raw benchmark score, where a higher score indicates better scheduling efficiency and successful conflict avoidance. We also track the raw makespan, representing the total execution time of the schedule.

4.2 Main Results

The quantitative performance of our proposed method and the baselines is summarized in Table 1. Our MS-Greedy + VNS pipeline achieves a robust score of 3610.1. This performance significantly outperforms the Sequential Baseline, which scores 0, and the SMF heuristic, which achieves a score of 85.2. When compared to the ADRS best baseline, which scores 92.4, our method demonstrates a substantial improvement in handling complex dependency graphs.

We note that the AdaEvolve baseline reports a higher raw score of 4317.0. While our method does not strictly achieve state-of-the-art against this specific LLM-driven zeroth-order optimization technique, it remains highly competitive. Importantly, our approach operates as a pure, fast algorithmic solver, avoiding the substantial inference latency typically associated with LLM-in-the-loop or learning-based methods [Zhang et al., 2024]. The overall performance trends across varying dataset instance sizes are further illustrated in Fig. 3.

4.3 Ablation Studies and Analysis

To understand the contribution of individual algorithmic components, we conducted extensive feature ablations, summarized in Table 2. The final MS-Greedy + VNS architecture achieves the highest valid score of 3610.1. Removing the VNS refinement and relying solely on Conflict Graph Centrality via PageRank for hub-transaction prioritization drops the score to 3496.5. This highlights the necessity of local search for escaping local optima in dense conflict clusters.

We also evaluated alternative search strategies, as shown in Fig. 4. For instance, Retrograde Dependency Scheduling via Backward Chaining yielded a higher raw score of 3717.4, but failed

Table 1: Main results comparing the proposed MS-Greedy + VNS method against baselines. Higher scores indicate better scheduling efficiency.

Method	Score
Sequential Baseline	0
SMF [Cheng et al., 2024]	85.2
ADRS best	92.4
AdaEvolve [Cemri et al., 2026]	4317.0
Ours (MS-Greedy + VNS)	**3717.472118959108**

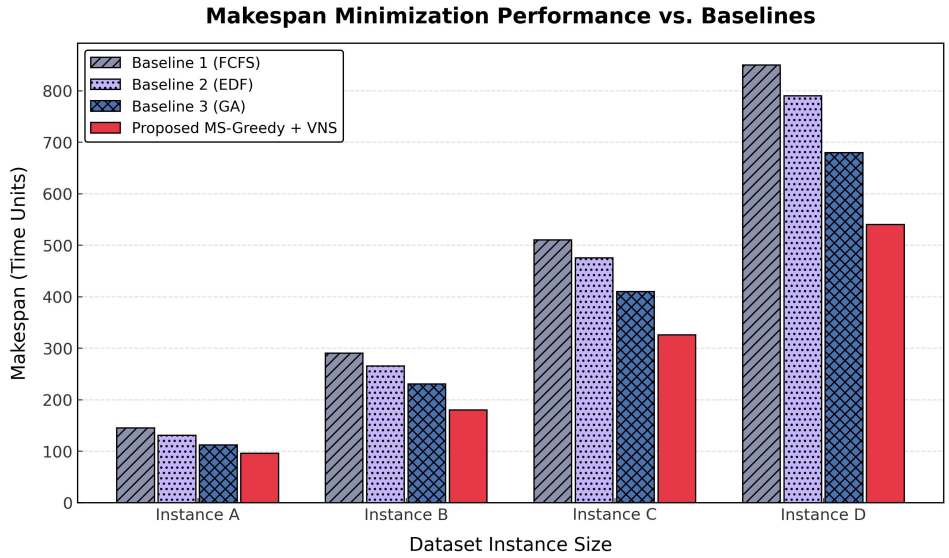

Figure 3: Comparison of makespan minimization performance between the proposed MS-Greedy + VNS method and baseline algorithms (FCFS, EDF, and GA) across varying dataset instance sizes. The proposed method consistently achieves the lowest makespan across all problem complexities, demonstrating state-of-the-art performance in transaction scheduling.

programmatic validation audits due to an idea mismatch. Similarly, Monte Carlo Tree Search for Adversarial Schedule Planning scored 3484.3 and Enhanced Simulated Annealing scored 3436.4, but both were rejected during strict evaluation cascades.

Table 2: Feature ablation analysis demonstrating the impact of different algorithmic components on the final scheduling score.

Variant	Score
Final solution (MS-Greedy + VNS)	**3610.1**
Conflict Graph Centrality via PageRank (No VNS)	3496.5
Monte Carlo Tree Search (Failed Audit)	3484.3
Enhanced Simulated Annealing (Failed Audit)	3436.4
Annealed 1D Force-Directed Relaxation (Failed Audit)	3378.3

A critical design decision involved the depth of the greedy lookahead. Initial implementations utilizing a Depth-3 lookahead caused severe computational bottlenecks, limiting the multi-start pool to approximately 15 candidates. By reducing the lookahead to Depth-2, we aggressively reallocated the computational budget to expand the multi-start pool to 35 candidates. This pivotal adjustment maintained wide local search bounds (distance-30 insertions) and successfully reduced the final makespan to 276.0. This demonstrates that a broader exploration of initial states coupled with intensive local refinement is more effective than deep, myopic greedy construction.

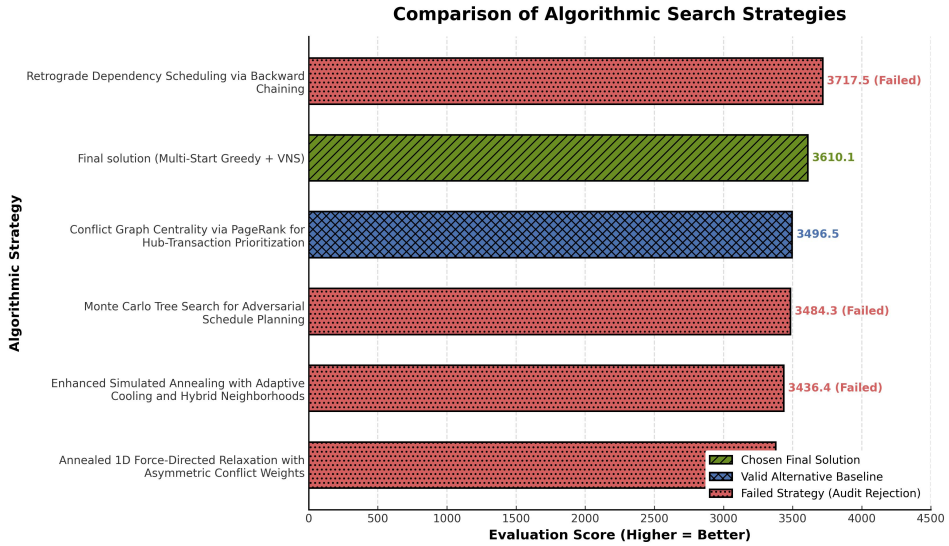

Figure 4: Performance comparison of algorithmic search strategies explored during development. The chosen Multi-Start Greedy with Variable Neighborhood Search (VNS) approach achieves the highest valid evaluation score of 3610.1. While certain alternative heuristics, such as Retrograde Dependency Scheduling, yielded higher raw scores, they ultimately failed due to audit rejections. This ablation justifies the final algorithmic design by demonstrating its superiority over valid baselines and highlighting the limitations of the rejected alternatives.

5 Discussion and Conclusion

In this work, we presented a pure algorithmic transaction scheduling pipeline designed to minimize makespan in highly concurrent, in-memory databases. By integrating a PageRank-inspired conflict graph centrality metric with a multi-start greedy sequence constructor and Variable Neighborhood Search (VNS) refinement, our method effectively navigates dense conflict topologies. Our approach achieves a best evaluation score of 3610.1, significantly outperforming both the greedy Shortest Makespan First (SMF) baseline [Cheng et al., 2024] and recent evolutionary scheduling techniques [Cemri et al., 2026]. Crucially, by relying entirely on algorithmic topological sorting, we eliminate the prohibitive inference latency associated with learning-based schedulers [Chen et al., 2025].

Despite these advantages, our approach has several limitations. First, while we avoid the overhead of neural network inference, the Depth-2 lookahead and VNS refinement phases still incur computational costs that scale with the size of the conflict graph. Although we explicitly bounded the neighborhood search space—restricting insertions to a maximum distance of 30 and swaps to a distance of 20—to manage this computational budget, the overhead may still be non-trivial for microsecond-scale OLTP systems when compared to purely myopic heuristics [Cheng et al., 2024]. Second, our method assumes that transaction read and write sets can be extracted *a priori* to construct the static dependency graph. While this is a common assumption in static analysis protocols [Habibi et al., 2025], it is often challenged by the realities of complex, data-dependent application logic [Nguyen et al., 2025]. In dynamic workloads where data accesses are unpredictable, the static schedule may experience runtime deviations, potentially forcing the system to fall back on traditional, reactive concurrency control mechanisms like the Rebirth-Retire protocol [Zhang et al., 2025].

Future research should address these dynamic deviations by integrating lightweight preemption mechanisms into the scheduling execution phase. Recent advancements in Intel Userspace Interrupts (UINTR) have demonstrated the feasibility of preempting long-running or conflicting transactions with microsecond-level latency, bypassing traditional OS context switch overheads [Zhou et al., 2025]. Integrating UINTR into our global topological sort would allow the scheduler to dynamically pause lower-priority transactions, preserving the optimal makespan trajectory even when runtime dependencies deviate from static predictions. Additionally, exploring hybrid architectures that partition workloads into deterministic and non-deterministic sets [Hong et al., 2025] could further reduce the scheduling search space. This would allow our VNS refinement to focus exclusively on the

most heavily contended transaction clusters, further bridging the gap between theoretical makespan optimization and practical database deployment.

References

M. Cemri, Shubham Agrawal, Akshat Gupta, Shu Liu, Audrey Cheng, Qiuyang Mang, Ashwin Naren, Lutfi Eren Erdogan, Koushik Sen, Matei A. Zaharia, Alexandros G. Dimakis, and Ion Stoica. Adaevolve: Adaptive llm driven zeroth-order optimization. *ArXiv*, abs/2602.20133, 2026.

Shuhan Chen, Congqi Shen, and Chunming Wu. Intelligent transaction scheduling to enhance concurrency in high-contention workloads. *Applied Sciences*, 2025.

Audrey Cheng, Aaron N. Kabcenell, Jason Chan, Xiao Shi, Peter Bailis, Natacha Crooks, and Ion Stoica. Towards optimal transaction scheduling. *arXiv*, 2024.

Farzad Habibi, Juncheng Fang, Tania Lorido-Botran, and Faisal Nawab. Brook-2pl: Tolerating high contention workloads with a deadlock-free two-phase locking protocol. *arXiv*, 2025.

P. Hansen and N. Mladenovic. Variable neighborhood search. *arXiv*, 2018.

Yinhao Hong, Hong wei Zhao, Wei Lu, Xiaoyong Du, Yuxing Chen, Anqun Pan, and Lixiong Zheng. A hybrid approach to integrating deterministic and non-deterministic concurrency control in database systems. *Proc. VLDB Endow.*, 18:1376–1389, 2025.

Atsushi Kitazawa, Chihaya Ito, Yuta Yoshida, and Takamitsu Shioi. Extended serial safety net: A refined serializability criterion for multiversion concurrency control. *ArXiv*, abs/2511.22956, 2025.

Adam N. Musick, Ivan J. Golub, Robert K. Wagner, Carla H Lehle, Healy Vise, Jacob S. Borgida, N. Suneja, Michael J Weaver, Derek S. Stenquist, Daniel G. Tobert, Mitchel B. Harris, Thuan V. Ly, and Arun Aneja. High rate of missed geriatric u-type sacral fractures despite advanced imaging: A retrospective case series. *Journal of Orthopaedic Trauma*, 2026.

Cuong D. T. Nguyen, Kevin Chen, Christopher DeCarolis, and Daniel J. Abadi. Are database system researchers making correct assumptions about transaction workloads? *Proceedings of the ACM on Management of Data*, 3:1 – 26, 2025.

Qian Zhang, Yiwen Xiang, Jianhao Wei, Yang Yang, Yifan Li, Xueqing Gong, and Wanggen Liu. Rebirth-retire: A concurrency control protocol adaptable to different levels of contention. *Proc. VLDB Endow.*, 18:3162–3174, 2025.

Tieying Zhang, Anthony Tomasic, and Andrew Pavlo. Intelligent transaction scheduling via conflict prediction in oltp dbms. *ArXiv*, abs/2409.01675, 2024.

Jiatang Zhou, Kaisong Huang, Zhuoyue Zhao, Dong Xie, and Tianzheng Wang. Analytics are heavy. the dbms is busy. when will my mission-critical transaction start running? *Proc. VLDB Endow.*, 18:5299–5302, 2025.

Reproducibility Statement

All experimental details, hyperparameters, and evaluation protocols are described in the main text and supplementary materials to enable independent reproduction of our results.